

Complete Code Explanation — Satellite Image Analysis Project

AID323 — Parallel and Distributed Computing

PART 1: WHERE DOES THE DATA COME FROM?

Real Satellite Data

Real satellite imagery comes from satellites orbiting Earth at around 800 km altitude. The most famous free source is **Sentinel-2**, operated by the European Space Agency (ESA).

Sentinel-2 captures images of Earth in 13 spectral bands — meaning it photographs Earth in 13 different wavelengths of light, including light the human eye cannot see.

The two bands our project uses are: - **Band 4 (RED)** — visible red light, wavelength 665 nm - **Band 8 (NIR)** — near-infrared light, wavelength 842 nm (invisible to humans)

Healthy vegetation absorbs red light (uses it for photosynthesis) but reflects near-infrared strongly. This difference is what NDVI measures.

Why We Use Synthetic Data Instead

In our code we generate fake data instead of reading real satellite files:

```
auto nir = generate_band(H, W, 0.6f, 42u); // fake NIR
auto red = generate_band(H, W, 0.3f, 137u); // fake RED
```

We do this because: 1. Real satellite files are gigabytes in size 2. Downloading and reading them requires extra libraries (GDAL) 3. For a parallel computing course, the MATH is what matters, not the data source 4. Our generated values (NIR ~0.6, RED ~0.3) fall in the same range as real vegetation pixels

The parallel speedup results are 100% identical whether data is real or synthetic.

PART 2: THE LIBRARIES — WHAT EVERY #include DOES

In sequential.cpp

#include <iostream>

The standard input/output library. Gives us cout (print to screen) and cin (read input). Without this, we cannot print anything. cpp
cout << "Hello" << endl; // needs iostream

#include <vector>

Gives us vector<float> — a dynamic array that manages its own memory. Instead of manually writing: cpp float* data = (float*)malloc(H * W * sizeof(float)); // C-style, dangerous free(data); // must remember to free We write: cpp vector<float> data(H * W); // automatic, safe, no memory leaks When data goes out of scope, memory is freed automatically.

#include <cmath>

Mathematical functions. We need it for: cpp exp(x) // e^x — used in the Gaussian formula Without this, the Gaussian kernel cannot be computed.

#include <chrono>

High-resolution timing library. Used to measure exactly how long each stage takes. cpp auto t0 = steady_clock::now(); // ... do work ... double seconds = duration<double>(steady_clock::now() - t0).count(); steady_clock never goes backwards (unlike system clock) so it is safe for benchmarking.

#include <fstream>

File input/output. Used to write timing results to a text file: `cpp ofstream f("sequential_time.txt"); f << total_time << "\n";` This file is later read by the benchmark script and plotting script.

#include <algorithm>

Gives us standard algorithms on containers: `cpp min_element(data.begin(), data.end()) // find minimum value`
`max_element(data.begin(), data.end()) // find maximum value` `copy(src.begin(), src.end(), dst) // copy a range`

#include <numeric>

Gives us mathematical operations on containers: `cpp accumulate(data.begin(), data.end(), 0.0) // sum all values`
(used for mean)

#include <iomanip>

Controls how numbers are printed: `cpp setprecision(4) // print 4 decimal places: 0.3398` `setw(8) // pad to 8 characters` `width fixed // always show decimal point (not scientific notation)`

Additional in parallel.cpp

#include <cstring>

Gives us `memcpy` for fast memory copying. Used when building the halo buffer: `cpp memcpy(destination, source, num_bytes);`

#include <mpi.h>

The entire MPI (Message Passing Interface) library. This is not part of the C++ standard — it must be installed separately. It gives us every function that starts with `MPI_`: `cpp MPI_Init() // start MPI` `MPI_Comm_rank() // which process am I?`
`MPI_Comm_size() // how many processes total?` `MPI_Sendrecv() // send AND receive simultaneously`
`MPI_Gatherv() // collect data from all processes to one` `MPI_Reduce() // combine values from all processes`
(e.g. `find max`) `MPI_Barrier() // wait until ALL processes reach this point` `MPI_Wtime() // high-precision timer` `MPI_Finalize() // shut down MPI cleanly`

#include <omp.h>

The OpenMP header. Gives us: `cpp omp_get_max_threads() // how many threads are available` `omp_get_thread_num() // which thread am I? (0, 1, 2, ...)` The actual parallelism comes from the compiler directives (pragmas), not function calls.

The using statements

`using namespace std;`

Without this, every standard library call would need the prefix `std::`: `cpp std::cout << std::fixed << std::setprecision(4);` // without using namespace `std` `cout << fixed << setprecision(4);` // with it — much cleaner

`using namespace chrono;`

Same idea for the timing library: `cpp std::chrono::steady_clock::now() // without` `steady_clock::now() // with`

PART 3: THE ALGORITHMS — WHERE THEY COME FROM AND WHY WE CHOSE THEM

Algorithm 1: NDVI (Normalised Difference Vegetation Index)

Origin

Developed by NASA scientists Rouse et al. in 1974 using data from the first Earth Resources Technology Satellite (ERTS-1, later renamed Landsat-1). It is now the most widely used vegetation index in remote sensing worldwide.

The Formula

$$\text{NDVI} = (\text{NIR} - \text{RED}) / (\text{NIR} + \text{RED})$$

Why This Formula Works

Plant leaves contain chlorophyll, which: - **Absorbs** red light strongly (uses it for photosynthesis) - **Reflects** near-infrared light strongly (NIR passes through leaf cells)

So for healthy vegetation: NIR is HIGH, RED is LOW → NDVI is close to +1 For bare soil or water: $\text{NIR} \approx \text{RED}$ → NDVI is near 0 or negative

Surface Type	NIR	RED	NDVI
Dense green forest	0.8	0.1	+0.78
Sparse vegetation	0.5	0.3	+0.25
Bare soil	0.3	0.3	0.00
Open water	0.05	0.07	-0.17
Cloud	0.9	0.9	~0.00

The Code

```
float d = nir[idx] + red[idx];
ndvi[idx] = (d > 1e-6f) ? (nir[idx] - red[idx]) / d : 0.0f;
```

The $d > 1e-6f$ check prevents division by zero (which would happen if both bands are zero — very dark or missing pixels).

Why It's Good for Parallelisation

Every pixel is completely independent. Pixel (100, 200) does not need pixel (100, 201) to compute its NDVI. This is called “embarrassingly parallel” — trivially split across any number of cores.

Algorithm 2: Gaussian Smoothing (2D Convolution)

Origin

Named after mathematician Carl Friedrich Gauss (1777–1855). The bell-curve distribution he described is the mathematical basis of the filter. In image processing, Gaussian smoothing has been standard since the 1980s.

Why We Need It

Raw satellite imagery contains noise — random pixel variations caused by: - Sensor electronics (thermal noise) - Atmospheric scattering - Quantisation errors

Gaussian smoothing removes this noise while preserving edges and large features better than a simple average (which blurs everything equally).

The Gaussian Function (2D)

$$G(x,y) = (1 / 2\pi\sigma^2) \times e^{-(x^2 + y^2) / 2\sigma^2}$$

Where: - x, y = distance from the centre of the kernel - σ (sigma) = controls how much blurring: larger σ = more blur - e = Euler's number (~2.718)

We use $\sigma = 2.0$, which gives a smooth blur without losing too much detail.

How the Kernel is Built

```
vector<float> make_gaussian_kernel(int size, float sigma) {
    int r = size / 2;          // radius = 7 for a 15x15 kernel
    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++) {
            float x = i - r;    // distance from centre (-7 to +7)
            float y = j - r;
            kernel[i*size+j] = exp(-(x*x + y*y) / (2*sigma*sigma));
        }
    }
}
```

```

// Normalise: divide by sum so all weights add up to 1.0
for (auto& v : kernel) v /= sum;
}

```

The resulting kernel looks like a hill seen from above — highest weight in the centre (1.0 after normalisation), dropping smoothly to near 0 at the edges.

How Convolution Works

For each pixel in the image, we: 1. Place the 15×15 kernel centred on that pixel 2. Multiply each kernel value by the corresponding pixel value in the image 3. Sum all 225 results → this becomes the new pixel value

Image pixel neighbourhood:	Kernel weights:	Products:
[0.3 0.4 0.2 ...]	[0.01 0.02 0.01 ...]	[0.003 0.008 ...]
[0.5 0.6 0.3 ...]	[0.02 0.10 0.02 ...]	= [0.010 0.060 ...]
[0.2 0.3 0.4 ...]	[0.01 0.02 0.01 ...]	[0.002 0.006 ...]

SUM of all products = new pixel value

Why It's the Expensive Step

- Image: $4096 \times 4096 = 16,777,216$ pixels
- Kernel: $15 \times 15 = 225$ multiplications per pixel
- Total: $16,777,216 \times 225 = \mathbf{3,774,873,600}$ operations (~3.8 billion)

This is why the sequential version takes ~4 seconds and why parallelising it gives such a dramatic speedup.

Why It's Good for Parallelisation

Each output pixel only depends on a small neighbourhood (± 7 pixels) of the input — not on the entire image. This makes it easy to split across processes, with only a thin strip of shared data (the halo) at the boundaries.

PART 4: HOW MPI WORKS IN OUR CODE

What MPI Is

MPI (Message Passing Interface) is a standard for writing programs that run as multiple separate processes, each with their OWN memory, possibly on different computers (nodes) in a cluster.

Think of it like several people working on the same puzzle: - Each person has their own pile of pieces (own memory) - They must explicitly hand pieces to each other when needed (message passing) - There is no shared table (no shared memory between processes)

How We Start MPI

```

MPI_Init(&argc, &argv); // start MPI runtime
MPI_Comm_rank(MPI_COMM_WORLD, &rank); // what is my process number? (0,1,2,3...)
MPI_Comm_size(MPI_COMM_WORLD, &nprocs); // how many processes total?

```

`MPI_COMM_WORLD` means “all processes in this run”. `rank` is like a worker ID — process 0 is usually the “master”.

How We Split the Image

The image has $H=4096$ rows. With 4 MPI processes: `cpp int base_rows = H / nprocs; // 4096 / 4 = 1024 rows each`
`int remainder = H % nprocs; // 4096 % 4 = 0 (divides evenly)`
`int local_rows = base_rows + (rank < remainder ? 1 : 0);`
`int start_row = rank * base_rows + (rank < remainder ? rank : remainder);`

Result: - Process 0: rows 0–1023 (`local_rows = 1024, start_row = 0`) - Process 1: rows 1024–2047 (`local_rows = 1024, start_row = 1024`) - Process 2: rows 2048–3071 (`local_rows = 1024, start_row = 2048`) - Process 3: rows 3072–4095 (`local_rows = 1024, start_row = 3072`)

The remainder handling ensures that if the image doesn't divide evenly (e.g. 4097 rows, 4 processes), the first few processes get one extra row. This is called **load balancing**.

The Halo Exchange — The Most Important MPI Concept

When process 1 tries to smooth its top row (row 1024), it needs pixels from rows 1017–1030 (kernel radius = 7). But rows 1017–1023 belong to process 0!

Process 1 does not have those rows. So BEFORE smoothing, processes exchange their border rows with their neighbours.

BEFORE halo exchange:

```
Process 0 owns: [row 0 ... row 1023]
Process 1 owns: [row 1024 ... row 2047]
```

AFTER halo exchange:

```
Process 0 has: [row 0 ... row 1023] + bottom_ghost[row 1024-1030]
Process 1 has: top_ghost[row 1017-1023] + [row 1024 ... row 2047] + bottom_ghost[row 2048-2054]
```

The ghost/halo rows are stored in a separate buffer called `halo_buf`.

The MPI_Sendrecv Call

```
MPI_Sendrecv(
    local_ndvi.data(), // WHAT to send: my top rows
    halo * W,          // HOW MANY floats to send: 7 rows × 4096 pixels
    MPI_FLOAT,         // DATA TYPE
    up,                // SEND TO: my upper neighbour (rank-1)
    10,                // TAG for this send (like a label)

    halo_buf.data(),   // WHERE to receive: top of my halo buffer
    halo * W,          // HOW MANY floats to receive
    MPI_FLOAT,         // DATA TYPE
    up,                // RECEIVE FROM: same upper neighbour
    11,                // TAG to match (neighbour sends with tag 11)

    MPI_COMM_WORLD,    // communicator
    &status             // status info (we don't use it here)
);
```

`MPI_Sendrecv` does both sending and receiving in ONE call. This avoids deadlock (where process A waits for B to receive before sending, while B also waits for A).

`MPI_PROC_NULL` is used for boundary processes (rank 0 has no upper neighbour, rank N-1 has no lower neighbour). Sending to/receiving from `MPI_PROC_NULL` is a no-op — it silently does nothing.

MPI_Gatherv — Collecting Results

After all processes finish smoothing their slice, results are collected at rank 0:

```
MPI_Gatherv(
    local_smooth.data(), // each process sends its local result
    local_rows * W,      // how many floats I'm sending
    MPI_FLOAT,

    global_result.data(), // rank 0 receives all results here
    recvcnts.data(),      // array: how many floats from each process
    displs.data(),        // array: where in global_result each goes
    MPI_FLOAT,
    0,                    // root: rank 0 is the collector
    MPI_COMM_WORLD
);
```

`recvcnts` and `displs` handle the case where different processes have different numbers of rows (load imbalance case).

MPI_Reduce — Getting the True Execution Time

Each process has its own local timer. The correct parallel execution time is the MAXIMUM across all processes (the slowest one determines wall time):

```
double loc_total = t_end - t_start; // my local time
double max_total;
```

```

MPI_Reduce(
    &loc_total,    // each process contributes their time
    &max_total,    // result stored here (only at rank 0)
    1,            // one value per process
    MPI_DOUBLE,
    MPI_MAX,      // operation: take the maximum
    0,            // store result at rank 0
    MPI_COMM_WORLD
);

```

MPI_Barrier — Synchronisation

```
MPI_Barrier(MPI_COMM_WORLD);
```

All processes stop here and WAIT until every single process reaches this line. Only then do they all continue. Used before starting the timer to ensure all processes begin at exactly the same moment.

PART 5: HOW OPENMP WORKS IN OUR CODE

What OpenMP Is

OpenMP (Open Multi-Processing) is a way to use multiple CPU threads within a single process. All threads share the SAME memory — so no message passing needed.

Think of it like one person with multiple hands — they share the same brain (memory) and can work on different parts of the problem simultaneously.

The Pragma Directive

```

#pragma omp parallel for schedule(static)
for (int i = 0; i < num_rows; i++) {
    // this loop body runs on multiple threads simultaneously
}

```

`#pragma omp parallel for` tells the compiler: “Split this loop’s iterations across all available threads.”

Without OpenMP, the loop runs on 1 thread: Thread 0: $i=0$, $i=1$, $i=2$, ... $i=1023$ (all 1024 rows, sequentially)

With OpenMP and 4 threads, each thread handles 256 rows: - Thread 0: $i=0$ to $i=255$ - Thread 1: $i=256$ to $i=511$ - Thread 2: $i=512$ to $i=767$ - Thread 3: $i=768$ to $i=1023$

All 4 threads run simultaneously on 4 CPU cores.

schedule(static)

`schedule( static )` means: divide the iterations evenly before the loop starts, and assign fixed chunks to each thread. Thread 0 always gets the first chunk, thread 1 always gets the second, etc.

This is the best choice here because: - Every row takes the same amount of time (equal workload) - No overhead from dynamic work assignment

The alternative `schedule( dynamic )` assigns work on-demand (better when iterations have unequal workloads, like in some graph algorithms).

OMP_NUM_THREADS

```
OMP_NUM_THREADS=4 mpirun -np 4 ./parallel
```

This environment variable tells OpenMP how many threads to use per process. Each MPI process creates 4 OpenMP threads → total cores used = $4 \times 4 = 16$.

Why Combine MPI and OpenMP?

Using ONLY MPI on a single multi-core machine wastes memory: - 8 MPI processes each load a copy of MPI metadata and runtime → ~8× overhead - Each process's memory is separate → no sharing possible

Using ONLY OpenMP cannot span multiple computers.

Hybrid MPI + OpenMP is the best of both: - MPI handles communication BETWEEN machines (distributed memory) - OpenMP handles parallelism WITHIN each machine (shared memory) - Less MPI overhead, better cache usage, can scale to thousands of cores

PART 6: THE FULL CODE FLOW — STEP BY STEP

sequential.cpp Flow

START

- Print configuration (image size, kernel size, memory)
- `make_gaussian_kernel(15, 2.0)`
 - Build 15×15 weight grid using the Gaussian formula
 - Normalise so weights sum to 1.0
- `generate_band(H, W, 0.6, 42)` → NIR band (16M floats)
- `generate_band(H, W, 0.3, 137)` → RED band (16M floats)
 - Each row seeded by: `seed + row_index × 1000`
 - Values clipped to `[0.0, 1.0]`
- `compute_ndvi(nir, red, H, W)` → NDVI map (16M floats)
 - For every pixel: $(nir - red) / (nir + red)$
 - Time measured
- `apply_gaussian(ndvi, kernel, H, W)` → smoothed NDVI (16M floats)
 - For every pixel: weighted sum of 15×15 neighbourhood
 - Time measured (this is the slow step)
- `compute_stats(smoothed)`
 - min, max, mean using STL algorithms
- Print results and timing table
- Write total time to `sequential_time.txt`

END

parallel.cpp Flow

START

- `MPI_Init` → each of the P processes gets a rank (0 to P-1)
- Rank 0 prints configuration
- `MPI_Barrier` → wait until ALL processes are ready
- Each process computes its row range (`start_row`, `local_rows`)
- `make_gaussian_kernel()` → same kernel on every process
- `generate_band_rows(start_row, local_rows, ...)`
 - Each process generates ONLY its own rows
 - Same seed formula as sequential → identical pixel values
- `MPI_Barrier` → sync before NDVI timing
- `compute_ndvi_omp()` [OpenMP parallel for]
 - Threads split `local_rows` evenly

```

    Each thread computes NDVI for its chunk of rows
- MPI_Barrier → sync before halo exchange
- Build halo_buf: copy local_ndvi into centre
- MPI_Sendrecv (top halo exchange)
  Send my top 7 rows → rank-1
  Receive rank-1's bottom 7 rows → my top ghost
- MPI_Sendrecv (bottom halo exchange)
  Send my bottom 7 rows → rank+1
  Receive rank+1's top 7 rows → my bottom ghost
- Fill boundary ghosts by mirroring own edge rows
- MPI_Barrier → sync before convolution timing
- apply_gaussian_omp(halo_buf, ...) [OpenMP parallel for]
  Threads split local_rows evenly
  Each thread smooths its chunk using halo_buf (which has ghost rows)
- MPI_Gatherv → rank 0 collects all smoothed slices
- MPI_Reduce(MPI_MAX) → rank 0 gets the maximum time across all ranks
- Rank 0 prints results, timing, and appends to parallel_time.txt
- MPI_Finalize → clean shutdown
END

```

PART 7: THE OUTPUT — WHAT EVERYTHING MEANS

Sequential Output

Sequential Satellite Image Analysis (C++)

```

Image : 4096 x 4096 (16777216 pixels) ← total pixels in image
Kernel : 15 x 15 (sigma = 2.0)      ← smoothing kernel size
Memory : ~320 MB                    ← 5 arrays × 40962 × 4 bytes

```

```

[1/4] Generating synthetic satellite bands ... 0.790 s
[2/4] Computing NDVI ... 0.303 s ← fast (simple math)
[3/4] Applying Gaussian smoothing ... 3.897 s ← slow (billions of ops)
[4/4] Computing statistics ... done

```

Results	
NDVI min	: 0.2065
NDVI max	: 0.4724
NDVI mean	: 0.3398
NDVI time	: 0.303 s
Gauss time	: 3.897 s
Total	: 4.200 s

```

← least vegetated pixel
← most vegetated pixel
← average vegetation across image

← baseline for speedup calculation

```

NDVI values 0.2–0.5 represent moderate to healthy vegetation — exactly what you’d expect from an agricultural or semi-vegetated area.

Parallel Output

Parallel Satellite Analysis (MPI + OpenMP)

```
Image : 4096 x 4096
MPI   : 4 processes           ← how many MPI processes launched
OpenMP: 2 threads/process     ← OMP_NUM_THREADS value
Kernel: 15 x 15 (sigma = 2.0)
```

NDVI min : 0.2065	← same as sequential (correct!)
NDVI max : 0.4724	← proves parallel version is correct
NDVI mean : 0.3398	
NDVI comp : 0.048 s	← ~6× faster (4 proc × 2 threads)
Halo exch : 0.001 s	← very small: <0.1% of total
Gauss conv : 0.612 s	← ~6× faster
Total : 0.660 s	

Speedup = $4.200 / 0.660 = 6.36\times$ with 8 total cores. Efficiency = $6.36 / 8 = 79.5\%$ — very good for a parallel program.

parallel time.txt format

Component	Value
convolution time (s)	0.048000
NDVI time (s)	0.001200
halo exchange time (s)	0.660000
total wall time (s)	0.612000
OpenMP threads per process	2
MPI process count	4

Performance Graphs (from plot_results.py)

fig1_exec_time.png — Shows total time dropping as more processes/threads added. Should show a curve dropping steeply at first then flattening (diminishing returns).

fig2_speedup.png — Speedup = Sequential time / Parallel time. A perfect parallel program would show a straight diagonal line (linear speedup). Real programs fall below this line due to communication and serial sections.

fig3_efficiency.png — Speedup / total_cores × 100%. 100% = perfect efficiency (impossible in practice). Drops as more cores added (communication overhead grows).

fig4_comm_overhead.png — Halo exchange time / total time $\times 100\%$. Should stay small ($<5\%$) confirming our algorithm is compute-dominated and communication is not the bottleneck.

PART 8: THE PROCESSORS — HOW MANY AND WHY

How Many Processors/Cores Are Used

When you run: `bash OMP_NUM_THREADS=2 mpirun -np 4 ./parallel`

You get: - 4 MPI processes - Each process spawns 2 OpenMP threads - Total cores used: $4 \times 2 = 8$ **cores**

How to Choose the Right Numbers

```
nproc          # shows total cores on your machine
```

If you have 8 cores, recommended configurations:

MPI processes	OMP threads	Total cores	Notes
1	1	1	baseline
1	8	8	pure OpenMP
2	4	8	hybrid
4	2	8	hybrid (our default)

MPI processes	OMP threads	Total cores	Notes
8	1	8	pure MPI

For your performance analysis, run ALL of these and compare. Typically: - 4 MPI × 2 OMP outperforms 8 MPI × 1 OMP (less communication overhead) - 1 MPI × 8 OMP is fastest on a single machine (no MPI overhead at all) - Multi-node cluster: MPI across nodes + OMP within each node is optimal

The MPI Rank System

```
mpirun -np 4 ./parallel
```

Launches 4 completely separate copies of the program. Each has: - Its own variables - Its own stack and heap - Its own rank number (0, 1, 2, 3)

Rank 0 is special — it is the one that: - Prints the final output - Gathers all results (MPI_Gatherv) - Writes the timing file

All ranks participate equally in: - Data generation - NDVI computation - Halo exchange - Gaussian smoothing

PART 9: THE COMMANDS — EVERY FLAG EXPLAINED

Compile Commands

```
g++ -O2 -std=c++17 -o sequential sequential.cpp -lm
```

- `g++` → GNU C++ compiler
- `-O2` → Optimisation level 2: enables loop unrolling, inlining, branch prediction hints. Makes code 2-4× faster.
- `-std=c++17` → Use C++ 2017 standard. Required for some modern features like structured bindings and improved algorithms.
- `-o sequential` → Name the output file “sequential” (default would be “a.out”)
- `sequential.cpp` → Source file to compile
- `-lm` → Link the math library. Required for `exp()`, `sqrt()`, etc.

```
mpicxx -O2 -std=c++17 -fopenmp -o parallel parallel.cpp -lm
```

- `mpicxx` → MPI C++ compiler wrapper. Internally calls `g++` but automatically adds `-I` and `-L` flags for MPI headers/libraries. Without it, you'd need to type long include paths manually.
- `-fopenmp` → Enable OpenMP. Without this flag, ALL `#pragma omp` lines are silently ignored — the code compiles but runs single-threaded.

Run Commands

```
./sequential
```

- `./` → Run from the current directory (required on Linux for local files)

```
OMP_NUM_THREADS=2 mpirun -np 4 ./parallel
```

- `OMP_NUM_THREADS=2` → Environment variable. Tells OpenMP runtime to use 2 threads per process. Must be set BEFORE `mpirun`.
- `mpirun` → MPI job launcher. Starts multiple processes.
- `-np 4` → Number of processes (np = number of processes).
- `./parallel` → The program each process runs.

```
mpirun --oversubscribe -np 16 ./parallel
```

- `--oversubscribe` → Allow more processes than physical cores. Normally MPI refuses to start if `np > nproc`. Use for testing when you don't have enough cores.

```
python3 plot_results.py
```

- python3 → Run with Python 3 (not Python 2, which is obsolete)
 - Script reads sequential_time.txt and parallel_time.txt, calls matplotlib to generate PNG graphs, saves them to results/ directory.
-

SUMMARY TABLE

Concept	What it is	Why we use it
NDVI	Vegetation index formula	Standard satellite science metric
Gaussian smoothing	Weighted neighbourhood average	Remove sensor noise
MPI	Multi-process parallelism	Distribute work across nodes/processes
OpenMP	Multi-thread parallelism	Use all cores within each node
Halo exchange	Border row communication	Needed for correct convolution at boundaries
vector<float>	Safe dynamic array	Avoids manual memory management
chrono	High-res timing	Accurate performance measurement
MPI_Sendrecv	Simultaneous send+receive	Deadlock-free halo exchange
MPI_Gatherv	Variable-count collection	Handles uneven row distribution
MPI_Reduce(MAX)	Find slowest process	True wall-clock time measurement
#pragma omp parallel for	Loop parallelisation	Split row processing across CPU threads
schedule(static)	Even work division	Optimal for uniform workloads
-O2	Compiler optimisation	2-4× faster without code changes
-fopenmp	Enable OpenMP	Without this, pragmas are ignored
mpicxx	MPI-aware compiler	Automatically includes MPI headers